

Chatbot Gaviota MVP: asistente conversacional para turismo gastronómico y hotelero en Cuba

SYALIA SRL

2026-04-21

1. Resumen

El Chatbot Gaviota MVP es un asistente conversacional Telegram (`gaviota_bot`) construido como prueba de concepto durante un curso impartido en Gaviota S.A., conjunto entre la Universidad de Alicante (GPLSI) y la Universidad de La Habana (GIA-UH), con apoyo de SYALIA SRL. **El sistema no tiene propósitos comerciales, sólo demostrativos**, y sirve como artefacto pedagógico para ilustrar la integración de modelos de lenguaje, búsqueda vectorial, agentes con herramientas y razonamiento estructurado en un dominio aplicado: el turismo gastronómico y hotelero en Cuba.

El asistente es multilingüe por demanda —responde en el idioma original del usuario— y opera sobre un inventario de hoteles, restaurantes y lugares indexado localmente como datos en formato JSON. Puede ser utilizado con cualquier modelo de lenguaje. Por defecto, se construyó usando como referencia al modelo Cecilia 2B, versión 1.0 creado por la Universidad de La Habana y la Universidad de Alicante; y el modelo Salamandra 2B, creado por el Barcelona Supercomputing Center, respectivamente.

2. Arquitectura del sistema

El sistema se organiza en cuatro capas. La capa de Telegram (`chatbot/app.py`) se construye sobre `python-telegram-bot` y mantiene un ciclo de sondeo con dos manejadores (`/start` y mensaje libre); un envoltorio `ConversationHandler` persiste el historial conversacional de cada usuario. La capa de agente (`chatbot/bot.py`) está construida sobre Lingo AI: materializa el agente, registra las habilidades decoradas con `@chatbot.skill` e implementa el pipeline de razonamiento estructurado (sección 5) y el conjunto de herramientas de inventario. La capa de datos se apoya en BeaverDB sobre SQLite local; almacena las colecciones de inventario indexadas vectorialmente, así como las colecciones por usuario `conversation:{user_id}` y `configs:{user_id}`. Finalmente, la capa de modelos es dual: un modelo de lenguaje conversacional servido por OpenRouter (Gemini 2.5 Flash por defecto, intercambiable) y un modelo de embeddings servido localmente vía LM Studio (`text-embedding-nomic-embed-text-v1.5` por defecto). La separación entre proveedor del modelo de lenguaje y proveedor de embeddings es deliberada: cada uno se reemplaza independientemente sin tocar la lógica del agente.

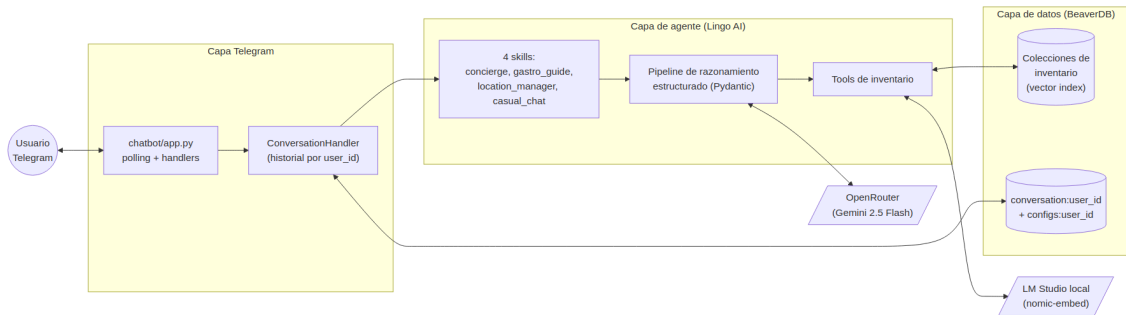


Figura 1: Arquitectura del Chatbot Gaviota MVP por capas.

3. Modelo de datos e indexación

El inventario consultable —hoteles, restaurantes, lugares— vive como archivos JSON bajo `data/jsons/`, uno por colección; el nombre del archivo determina el nombre de la colección BeaverDB resultante. El script `chatbot/index.py` recorre cada archivo y, para cada registro (con campos esperados `name` y `description`), construye un texto de la forma `f"{name}. {description}"`, lo embebe con el modelo configurado y almacena un Document con dos partes: el vector de embedding y el cuerpo completo del registro preservado tal cual.

Esa separación es deliberada e importante. El embedding sirve exclusivamente para recuperación semántica —búsqueda por descripción libre, similitud de propósito—, mientras que los atributos detallados (precio, dirección, teléfono, tipo de cocina, barrio, capacidad) viven en el cuerpo sin pérdida y se consultan en una fase posterior de enriquecimiento sobre los candidatos ya recuperados. Esta arquitectura permite respuestas que combinan recuperación difusa con datos estrictos. La memoria conversacional usa una colección separada `conversation:{user_id}` —una lista de mensajes Pydantic— con configuración por usuario en `configs:{user_id}`; la ventana de contexto efectiva es configurable (cinco mensajes por defecto).

4. Habilidades y enrutamiento

El agente expone cuatro habilidades decoradas con `@chatbot.skill`, que Lingo enruta a partir del mensaje del usuario y del contexto conversacional acumulado:

- `concierge` — consultas generales de turismo: qué hacer, planes mixtos y recomendaciones cruzadas que combinan hoteles, restaurantes y lugares.
- `gastro_guide` — específica de gastronomía: búsqueda y filtrado de restaurantes, planificación de menús multi-día y comparación por cocina, precio o barrio.
- `location_manager` — conciencia geográfica: dónde está el usuario, búsqueda por proximidad, resolución semántica de menciones libres a municipios cubanos contra `CUBAN_GEOGRAPHY` (un mapeo provincia → municipios codificado directamente en `chatbot/utls.py`) y verificación de la vigencia de la ubicación declarada.
- `casual_chat` — conversación de respaldo para turnos no clasificables como solicitud de inventario.

Cada habilidad ejecuta el pipeline de razonamiento descrito en la sección 5 e invoca el subconjunto de herramientas relevantes. Por ejemplo, `gastro_guide` emplea `search_restaurants_by_description`, `filter_restaurants` y `get_restaurant_details`; `location_manager` usa `find_places_location` y `set_user_location`. Las herramientas son funciones asíncronas tipadas que el agente puede invocar como parte de su plan de ejecución.

5. Pipeline de razonamiento estructurado

Dentro de cada habilidad, el procesamiento de un turno se organiza en cinco etapas, cada una con salidas tipadas mediante Pydantic, y no como una sola llamada al modelo de lenguaje:

1. Extracción de la intención. La función `get_user_intent` invoca al modelo para producir un `UserIntent` tipado: tipo de petición, alcance del contexto (`ContextScope`: nuevo, refinamiento, paginación) y entidades mencionadas. En paralelo, `get_search_limit` extrae cuántos resultados pidió explícitamente el usuario.
2. Intención espacial. Cuando aplica, `get_spatial_intent` produce un `SpatialIntent` con el tipo de relación geográfica (en, cerca de, entre) y el tipo de ancla (municipio, hito, ubicación previa del usuario). `resolve_municipality_semantic` traduce menciones libres al `CUBAN_GEOGRAPHY` codificado en el código fuente.
3. Diseño del plan. La función `design_data_processing_plan` genera un `StrategicPlan` con una lista de `ResourceActions` (qué colecciones consultar, con qué filtros) y una `ProcessingRecipe` con `ProcessSteps` que describen cómo combinar, ordenar y limitar los resultados antes de pasarlos al modelo final.
4. Ejecución de herramientas. Las acciones del plan disparan búsqueda vectorial (`_vector_search` sobre la colección correspondiente), filtros estructurados sobre el cuerpo completo —apoyados en primitivas de coincidencia difusa: `is_fuzzy_match`, `check_any_match`, `check_text_match`, `count_matches`— y enriquecimiento por nombre exacto cuando el usuario menciona una entidad concreta.
5. Síntesis final. El modelo compone la respuesta al usuario con un prompt de sistema aumentado por el `ANTI_HALLUCINATION_GUARD`. Esa salvaguarda establece una jerarquía explícita: los resultados filtrados por las herramientas (`TOOLS_EXECUTION_LOG`) son la lista canónica; el inventario completo (`INVENTORY_DATA`) es expansión permitida sólo cuando el registro está vacío o es insuficiente; queda prohibido mezclar atributos entre entidades; y los datos faltantes deben omitirse en silencio, no rellenarse con etiquetas del tipo “no disponible”.

El estado de búsqueda —consulta, límite, candidatos— se persiste vía `save_search_snapshot`, lo cual habilita paginación y refinamiento natural a lo largo de turnos sucesivos: “muéstrame más”, “filtra los anteriores por barrio Vedado”, “el segundo de la lista, dame su dirección”.

6. Capacidades demostradas

El bot real (@gaviota_bot en Telegram, durante las sesiones del curso) muestra tres rasgos relevantes en interacción con usuarios:

- Multilingüe por demanda. Cuando el usuario cambia al francés a media conversación, el asistente detecta el cambio y continúa la interacción en francés sin perder el contexto previo, traduciendo descripciones, barrios y rangos de precio.
- Conocimiento estructurado del inventario. Las recomendaciones incluyen consistentemente atributos verificables del JSON indexado: nombre del establecimiento, barrio (Vieille Havane, Plaza de la Revolución, Vedado, etcétera), tipo de cocina y rango de precios en USD, sin alucinar campos ausentes, gracias al `ANTI_HALLUCINATION_GUARD`.
- Planificación multi-día. Ante una solicitud abierta como “planifícame una semana de cenas con variedad”, el asistente estructura una propuesta día a día con criterios cruzados: estilo de cocina distinto cada día, distribución geográfica entre barrios y rangos de precio coherentes, apoyándose en el `StrategicPlan` para coordinar varias búsquedas en una sola respuesta.

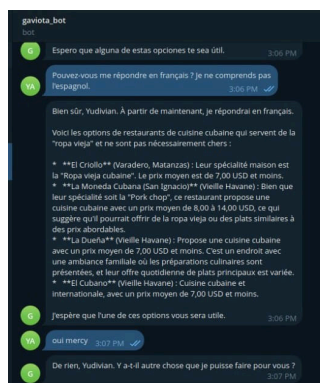


Figura 2: Conversación multilingüe: el asistente continúa en francés.

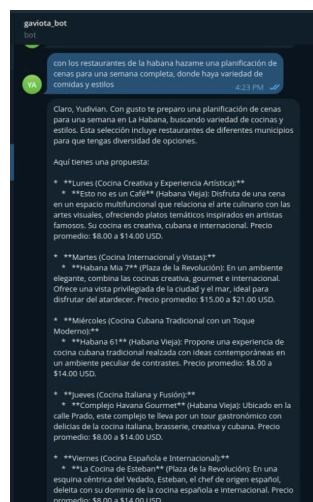


Figura 3: Planificación de cenas para una semana en La Habana.

7. Limitaciones y aprendizajes

El sistema no incorpora autenticación más allá del modelo de Telegram, depende de un proveedor de modelo de lenguaje externo (OpenRouter) y de un servicio de embeddings local (LM Studio); la geografía cubana está codificada directamente en `chatbot/utls.py` en lugar de provenir de una fuente estructurada actualizable; no hay pruebas automatizadas ni integración continua; el inventario JSON requiere curación manual y reindexación explícita vía `chatbot/index.py` cada vez que cambia. Estas limitaciones son consistentes con la naturaleza de prueba de concepto del proyecto y no se plantean como deuda técnica a saldar.

Como artefacto pedagógico, el PoC ilustra patrones extrapolables a otros dominios verticales: la separación entre recuperación semántica (embedding sobre nombre y descripción) y enriquecimiento estructurado (cuerpo completo en BeaverDB); el uso de salidas Pydantic tipadas como contrato entre etapas del razonamiento, en lugar de prompts monolíticos; y la formalización de una salvaguarda contra la alucinación con jerarquía de fuentes y prohibición explícita de mezclar atributos entre entidades. Cualquier reutilización futura por parte de SYALIA partiría de estos patrones como aprendizajes —no del MVP como producto— y supondría reescritura desde cero del inventario, los prompts y la capa de habilidades en función del nuevo dominio.